

## Code Explanation

### DPAAddressChangeYUYU Data Processing Module Explanation

The DPAAddressChangeYUYU data processing module is designed to process address change data and generate the DPAAddressChangeYUYU output file. This module implements an end-to-end workflow that includes checking source records, processing data, and writing output files.

#### ## Overview of the Code

The code is written in Python and utilizes the PySpark library to process data. It defines a class `DPAAddressChangeProcessor` that encapsulates the data processing logic. The main workflow is executed through the `run\_workflow` function, which creates an instance of `DPAAddressChangeProcessor` and calls its methods to perform the necessary operations.

#### ## Step 1: Initialization of DPAAddressChangeProcessor

The `DPAAddressChangeProcessor` class is initialized with a `SparkSession` and a configuration dictionary. The configuration dictionary contains settings such as the target directory for output files.

```
class DPAAddressChangeProcessor:
    def __init__(self, spark: SparkSession, config: Dict[str, str]):
        self.spark = spark
        self.config = config
        self.timestamp = datetime.now().strftime("%Y%m%d")
```

#### ## Step 2: Checking Source Records

The `check\_source\_records` method checks if source records exist in the staging table `STG\_E2E\_AC\_TXDBH\_DATA`. It returns the count of records in the source table and writes the count to an audit file.

```
def check_source_records(self) -> int:
    df = self.spark.table("STG_E2E_AC_TXDBH_DATA")
    count_df = df.agg(count("*").alias("record_count"))
    record_count = count_df.collect()[0]["record_count"]

    count_df.coalesce(1).write
        .mode("overwrite")
        .option("header", "true")
        .csv(f"{self.config['target_dir']}/FF_Source_Count.out")

    return record_count
```

#### ## Step 3: Processing YUYU Creation

The `process\_yuyu\_creation` method is the main processing logic for YUYU creation. It reads source data, applies single-byte to full-width Katakana transliteration, performs lookups for owner information, applies transformations, generates sequence numbers, and prepares the final YUYU output DataFrame.

```
def process_yuyu_creation(self) -> Tuple[DataFrame, DataFrame]:
    source_df = self.spark.sql("""
        SELECT
```

```

        *
        /
        CAST(PPAY_ADR1 AS STRING) AS PPAY_ADR1_FW,
        CAST(PPAY_ADR2 AS STRING) AS PPAY_ADR2_FW,
        CAST(PPAY_ADR3 AS STRING) AS PPAY_ADR3_FW
    FROM STG_E2E_AC_TXDBH_DATA
    """
)

    transliterate_udf = udf(self._transliterate_to_fullwidth, StringType())

    source_df = source_df
    .withColumn("PPAY_ADR1_FW", transliterate_udf(col("PPAY_ADR1")))
    .withColumn("PPAY_ADR2_FW", transliterate_udf(col("PPAY_ADR2")))
    .withColumn("PPAY_ADR3_FW", transliterate_udf(col("PPAY_ADR3")))

    source_with_lookups = self._apply_lookups(source_df)
    transformed_df = self._apply_transformations(source_with_lookups)
    with_sequence = self._add_sequence_numbers(transformed_df)
    yuyu_output_df = self._prepare_yuyu_output(with_sequence)
    staging_target_df = source_df

    return yuyu_output_df, staging_target_df

```

#### ## Step 4: Writing YUYU Output and Staging Target

The `write_yuyu_output` and `write_staging_target` methods write the YUYU output DataFrame and staging target DataFrame to their respective destinations.

```

def write_yuyu_output(self, df: DataFrame) -> None:
    output_path = f"{self.config['target_dir']}/dpaddresschange_yuyu_{self.timestamp}.out"

    df.coalesce(1).write
        .mode("overwrite")
        .option("header", "false")
        .csv(output_path)

def write_staging_target(self, df: DataFrame) -> None:
    df.write
        .mode("overwrite")
        .option("mergeSchema", "true")
        .saveAsTable("STG_E2E_AC_TXDBH_DATA_TGT")

```

#### ## Step 5: Running the Workflow

The `run_workflow` function creates an instance of `DPAddressChangeProcessor` and calls its methods to execute the workflow.

```

def run_workflow(spark: SparkSession, config: Dict[str, str]) -> None:
    processor = DPAddressChangeProcessor(spark, config)

```

```
record_count = processor.check_source_records()

if record_count == 0:
    processor.create_empty_trigger_file()
else:
    yuyu_df, staging_df = processor.process_yuyu_creation()
    processor.write_yuyu_output(yuyu_df)
    processor.write_staging_target(staging_df)
    processor.delete_trigger_file()
```